SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE:ITA0612 COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO3: Bayes Theorem – Concept Learning – Maximum Likelihood – Minimum Description Length Principle – Bayes Optimal Classifier – Gibbs Algorithm – Naïve Bayes Classifier – Bayesian Belief Network – EM Algorithm – Probability Learning – Sample Complexity – Finite and Infinite Hypothesis Spaces – Mistake Bound Model, (BL 6)

Assessment Tool Weightage: 40%

QUESTIONS: 15 **Total Marks:25**

Annexure A

Experiments

Code of Conduct:

I K Pujitha (192524149) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *K Pujitha*

Annexure A

Short Answer Test

1. Design and conduct an experiment to implement a Naïve Bayes classifier on a real-world dataset. Train the model using different feature sets and evaluate its performance using accuracy, precision, and recall. Analyze how assumptions of feature independence affect the results. Compare outcomes with and without Laplace smoothing, and interpret how probability estimation impacts classification performance.

2. Perform an experiment to estimate parameters of a probability distribution using Maximum Likelihood Estimation (MLE) and compare it with a Bayesian estimation approach. Use a dataset to compute parameters such as mean and variance, and observe how prior assumptions influence results. Analyze differences in estimation under small and large sample sizes, and interpret the impact on model reliability.

3. Implement the Expectation-Maximization (EM) algorithm for a mixture model and conduct experiments to study its convergence behavior. Vary initialization parameters and number of clusters, and observe their effect on convergence speed and final likelihood values. Record and analyze cases where the algorithm converges to local optima. Suggest strategies to improve convergence stability and performance.

4. Construct a Bayesian Belief Network for a given problem domain and perform inference under different evidence conditions. Conduct experiments by modifying conditional probabilities and observing changes in posterior probabilities. Analyze how dependencies between variables influence the results. Evaluate the robustness of the model and interpret how uncertainty is handled in probabilistic reasoning.

5. Implement a concept learning algorithm (such as Candidate Elimination) and perform experiments to analyze the effect of hypothesis space size on learning performance. Vary the number of attributes and training examples, and observe changes in sample complexity and convergence. Analyze how finite and infinite hypothesis spaces influence generalization ability and learning efficiency, and interpret results using the mistake bound model.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**15**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**15**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

1. Naïve Bayes Classifier

Aim:

To implement a Naïve Bayes classifier using a real-world dataset and evaluate its performance using accuracy, precision and recall.

Explanation:

Naïve Bayes is a supervised machine learning algorithm based on Bayes' Theorem. It is mainly used for classification problems such as spam detection, disease prediction and sentiment analysis. It calculates the probability of a particular class based on the given input features.

Procedure:

1. Select a real-world dataset, such as the Iris dataset or a spam email dataset.
2. Preprocess the data by removing missing values and selecting important features.
3. Divide the dataset into training and testing sets.
4. Train the Naïve Bayes classifier using different feature combinations.
5. Predict the output for the testing data.
6. Calculate accuracy, precision and recall to evaluate performance.
7. Compare the results with and without Laplace smoothing.

Performance Evaluation:

- Accuracy: Measures the percentage of correctly classified samples.
- Precision: Measures how many predicted positive samples are actually positive.
- Recall: Measures how many actual positive samples are correctly identified.

Feature Independence: Naïve Bayes assumes that all features are conditionally independent given the class. However, in real-world datasets, some features may be related. This assumption can affect prediction accuracy.

Laplace Smoothing: It is used to avoid zero probability when a feature does not appear in the training data. It adds a small value (usually 1) to the frequency counts, making probability estimation more reliable.

Result:

Naïve Bayes provides fast and effective classification. Feature selection and Laplace smoothing help improve its reliability and performance.

2. Maximum Likelihood Estimation (MLE) and Bayesian Estimation

Aim:

To estimate the parameters of a probability distribution using MLE and Bayesian estimation and compare their performance.

Explanation:

Maximum Likelihood Estimation and Bayesian Estimation are two important methods used to estimate unknown parameters in probability models. Parameters such as mean and variance help describe the distribution of data.

Procedure:

1. Select a numerical dataset containing values such as student marks or customer ages.
2. Calculate the sample mean and variance.
3. Apply MLE to estimate the distribution parameters.
4. Apply Bayesian estimation by selecting suitable prior distributions.
5. Compare both methods using small and large sample sizes.
6. Observe how prior assumptions influence the estimated parameters.

Maximum Likelihood Estimation (MLE):

MLE estimates the parameters by maximizing the likelihood of observing the given data.

For a normal distribution:

$$\mu_{MLE}=\frac{1}{n}\sum_{i=1}^{n}x_i$$
$$\sigma^2_{MLE}=\frac{1}{n}\sum_{i=1}^{n}(x_i-\mu)^2$$

Bayesian Estimation:

Bayesian estimation combines prior knowledge with observed data to calculate the posterior distribution.

$$ P(\theta|D)=\frac{P(D|\theta)P(\theta)}{P(D)} $$

Here, prior knowledge is represented by $P(\theta)$, and observed data is represented by D.

Analysis:

- MLE mainly depends on the observed data.
- Bayesian estimation uses both prior knowledge and observed data.
- For small datasets, prior assumptions can strongly influence Bayesian estimates.
- For large datasets, the influence of prior assumptions generally decreases.
- Both methods become more reliable when sufficient representative data is available.

Result:

MLE provides estimates based on observed data, whereas Bayesian estimation incorporates prior knowledge. Bayesian estimation is especially useful when the available dataset is small.

3. Expectation-Maximization (EM) Algorithm

Aim: To implement the Expectation-Maximization algorithm for a mixture model and study its convergence behavior.

Explanation:

The Expectation-Maximization (EM) algorithm is an unsupervised machine learning algorithm used to estimate unknown parameters when data contains hidden or missing information. It is commonly used in clustering and Gaussian Mixture Models (GMM).

Procedure:

1. Select a dataset containing customer spending, purchasing frequency or other numerical features.
2. Preprocess the dataset by handling missing values and normalizing the data.
3. Select the number of clusters and initialize their parameters.
4. Perform the Expectation step (E-step).
5. Perform the Maximization step (M-step).
6. Repeat both steps until the model converges.
7. Compare the final likelihood values using different initializations and cluster numbers.

Working of EM Algorithm:

1. Expectation Step (E-step):

The algorithm calculates the probability that each data point belongs to a particular cluster based on the current parameters. These probabilities are called responsibilities.

2. Maximization Step (M-step):

The algorithm updates the cluster parameters, such as mean, variance and mixing probabilities, using the responsibilities calculated in the E-step.

These two steps are repeated until the likelihood stops improving significantly.

Analysis:

- EM gradually improves the likelihood of the model.
- Different initializations may produce different clustering results.
- Increasing the number of clusters changes the model complexity.
- Poor initialization may cause the algorithm to converge to a local optimum instead of the global optimum.
- Multiple random initializations and suitable cluster selection can improve performance.

Applications:

EM can be used in customer segmentation, image processing, speech recognition and pattern recognition.

Result:

The EM algorithm effectively identifies hidden clusters and estimates their distributions. Its performance depends on initialization, number of clusters and data quality.

4. Bayesian Belief Network (BBN)

Aim:

To construct a Bayesian Belief Network and perform probabilistic inference under different evidence conditions.

Explanation:

A Bayesian Belief Network is a probabilistic graphical model that represents relationships and dependencies between different variables. It uses a directed acyclic graph (DAG) to represent these relationships and conditional probability tables to calculate probabilities.

Procedure:

1. Select a problem domain, such as disease diagnosis.
2. Identify variables such as Disease, Fever, Cough and Medical History.
3. Construct a directed graph showing the dependencies between variables.
4. Assign prior probabilities and conditional probabilities to each variable.
5. Provide different evidence conditions and calculate posterior probabilities.
6. Modify conditional probabilities and observe changes in predictions.
7. Evaluate the model's performance and reliability.

Example:

Consider a disease diagnosis system:

- Disease is the main variable.

- Fever and Cough are symptoms.
- Medical History can influence the probability of disease.

When a patient has fever and cough, the network calculates the probability of having the disease using the available evidence.

Analysis:

- Prior probabilities represent the initial possibility of an event.
- Conditional probabilities describe the relationship between dependent variables.
- Posterior probabilities are updated when new evidence is provided.
- Dependencies between symptoms can improve reasoning compared with assuming all symptoms are independent.
- Missing data and incorrect probability values may affect the model's predictions.

Advantages:

1. Handles uncertainty effectively.
2. Represents dependencies between variables.
3. Updates predictions when new evidence is available.
4. Supports decision-making in medical diagnosis and risk analysis.

Result:

Bayesian Belief Networks provide an effective method for probabilistic reasoning and decision-making under uncertainty.

5. Candidate Elimination Algorithm

Aim:

To implement the Candidate Elimination algorithm and analyze the effect of hypothesis space size on learning performance.

Explanation:

Candidate Elimination is a concept learning algorithm used in supervised machine learning. It learns concepts from positive and negative training examples by maintaining two boundaries: the Specific boundary (S) and the General boundary (G).

The hypothesis space contains all possible hypotheses that can describe a particular concept.

Procedure:

1. Select a training dataset containing attributes and labelled examples.
2. Initialize the Specific boundary (S) with the most specific hypothesis.
3. Initialize the General boundary (G) with the most general hypothesis.
4. Read the training examples one by one.
5. For positive examples, generalize the Specific boundary as required.
6. For negative examples, specialize the General boundary as required.
7. Continue updating both boundaries until all training examples are processed.
8. Analyze the final version space and learning performance.

Analysis:

Hypothesis Space: It represents all possible rules that the algorithm can consider. Increasing the number of attributes generally increases the size of the hypothesis space.

Finite Hypothesis Space: It contains a limited number of possible hypotheses. Learning and searching can be easier because the number of possible rules is restricted.

Infinite Hypothesis Space: It contains unlimited possible hypotheses. It may require more computation and additional restrictions to achieve effective learning.

Sample Complexity: It refers to the number of training examples required to learn a concept with acceptable accuracy. A larger hypothesis space generally requires more training examples.

Mistake Bound Model: It estimates the maximum number of prediction mistakes an algorithm may make under specific learning assumptions.

Applications:

Candidate Elimination can be used in student performance prediction, medical classification and pattern recognition.

Result:

Candidate Elimination identifies the set of hypotheses consistent with the training examples. Its learning efficiency depends on the hypothesis space, number of attributes and quality of training data.